Robot Arm Control System

Generated by Doxygen 1.9.8

Chapter 1

TÍMOVÝ PROJEKT

1.1 Vnorený riadiaci systém pre model robotického manipulátora

placeholder text - opis projektu

1.1.1 Riešitelia

- Bc. Simona Walterová - Team Leader
- Bc. Samuel Vavruš - Tech Leader
- Bc. Samuel Valko
- Bc. Samuel Kopp
- Bc. Márk Pšenák
- Bc. Filip Dubovský
- Bc. Jakub Janega

1.1.2 Vedúci projektu

- Ing. Martin Dodek, PhD.

1.1.3 Branches

- `main` - default branch
- `devel` - branch pre vývoj projektu
- `docs` - branch pre vývoj dokumentácie

1.1.4 Dokumentácia

`thesis.pdf`

Chapter 2

Namespace Index

2.1 Namespace List

Here is a list of all documented namespaces with brief descriptions:

Chapter 3

Hierarchical Index

3.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

Chapter 4

Class Index

4.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Chapter 5

File Index

5.1 File List

Here is a list of all documented files with brief descriptions:

Chapter 6

Namespace Documentation

6.1 Robot Namespace Reference

Core robotics control and kinematics namespace.

Classes

- class JointActuators

  Abstract base class for joint actuation hardware.
- class RobotArm

  High-level robot arm control interface.
- class RobotKinematics

  Concrete kinematics solver using Jacobian-based inverse kinematics.
- class RobotKinematicsBase

  Abstract base class for robot kinematics solvers.

6.1.1 Detailed Description

Core robotics control and kinematics namespace.
Contains base classes and implementations for robot arm kinematic control, including forward kinematics, inverse kinematics (IK), and joint actuation.

Chapter 7

Class Documentation

7.1 actuator_parameters Struct Reference

Joint actuator configuration and calibration parameters.
`#include <robot.h>`

Public Attributes

- float **minAngle**

  Minimum joint angle in radians.
- float **maxAngle**

  Maximum joint angle in radians.
- float **gain**

  Scaling factor for angle-to-PWM conversion.
- float **offset**

  Offset applied to joint angles during conversion.

7.1.1 Detailed Description

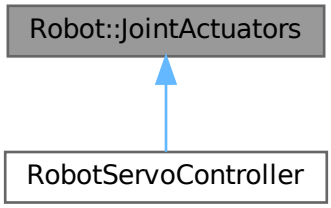
Joint actuator configuration and calibration parameters.
Stores calibration and limit parameters for a single joint actuator. Used to convert between joint angle space and PWM control signals.
The documentation for this struct was generated from the following file:

- TP26-Roboticke-rameno/robot.h

7.2 Robot::JointActuators Class Reference

Abstract base class for joint actuation hardware.
`#include <robot.h>`

Inheritance diagram for Robot::JointActuators:



Public Member Functions

- **JointActuators** ()

  Default constructor.
- virtual bool writeAngles (const float ∗, std::size_t)=0

  Sends joint angle commands to all actuators.

7.2.1 Detailed Description

Abstract base class for joint actuation hardware.
Defines the interface for sending computed joint angles to physical actuators (servos, motors, etc.). Implementations handle conversion from angles to hardware control signals (PWM, current commands, etc.).

7.2.2 Member Function Documentation

7.2.2.1 writeAngles()

```
virtual bool Robot::JointActuators::writeAngles (
            const float * ,
            std::size_t  )  [pure virtual]
```

Sends joint angle commands to all actuators.

Parameters

angles	Array of joint angles in radians
n	Number of joints (should match DOF)

Returns

true if all angles were within valid ranges and written successfully, false if any angle exceeded joint limits

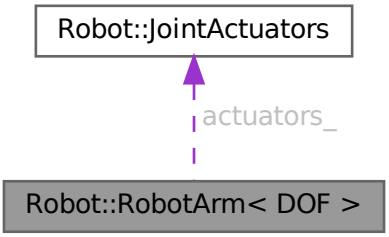
Implemented in RobotServoController.
The documentation for this class was generated from the following file:

- TP26-Roboticke-rameno/robot.h

7.3 Robot::RobotArm< DOF > Class Template Reference

High-level robot arm control interface.
`#include <robot.h>`

Collaboration diagram for Robot::RobotArm< DOF >:



Public Member Functions

- RobotArm (RobotKinematicsBase< DOF > *robot_kinematics, JointActuators *actuators)

  Initializes robot arm with kinematics solver and actuator hardware.
- ~**RobotArm** ()=default

  Destructor.
- bool setJointAngles (const Matrix< DOF, 1 > &angles)

  Sets and commits joint angles to hardware actuators.
- float getJointAngle (std::size_t index) const

  Gets current angle of a single joint.
- Matrix< DOF, 1 > getJointAngles () const

  Gets current angles of all joints.
- std::pair< Matrix< 3, 1 >, Matrix< 3, 3 > > getCurrentEffectorPositionOrientation () const

  Computes current end-effector pose from joint state.
- bool goTo (const Matrix< 3, 1 > &pr, const Matrix< 3, 3 > &Rr)

  Moves robot end-effector to desired position and orientation.

Protected Attributes

- JointActuators * **actuators_**

  Pointer to joint actuator hardware interface.
- Matrix< DOF, 1 > **jointAngles_**

  Current joint angles in radians.
- RobotKinematicsBase< DOF > * **robot_kinematics_**

  Pointer to kinematics solver.

7.3.1 Detailed Description

template<std::size_t DOF>
class Robot::RobotArm< DOF >

High-level robot arm control interface.

Template Parameters

DOF	Degrees of freedom (number of joints)

Coordinates kinematics solvers with hardware actuators to provide intuitive end-effector-centric control. Maintains

current joint state and manages both forward kinematics (state monitoring) and inverse kinematics (motion planning).

7.3.2 Constructor & Destructor Documentation

7.3.2.1 RobotArm()

```
template<std::size_t DOF>
Robot::RobotArm< DOF >::RobotArm (
            RobotKinematicsBase< DOF > * robot_kinematics,
            JointActuators * actuators )  [inline]
```

Initializes robot arm with kinematics solver and actuator hardware.

Parameters

robot_kinematics	Pointer to kinematics solver instance
actuators	Pointer to joint actuators instance

Does not take ownership of pointers; caller must manage object lifetimes.

7.3.3 Member Function Documentation

7.3.3.1 getCurrentEffectorPositionOrientation()

```
template<std::size_t DOF>
std::pair< Matrix< 3, 1 >, Matrix< 3, 3 > > Robot::RobotArm< DOF >::getCurrentEffector↩
PositionOrientation ( ) const  [inline]
```

Computes current end-effector pose from joint state.

Returns

Pair of (end-effector position [x,y,z] in meters, end-effector rotation matrix 3x3)

7.3.3.2 getJointAngle()

```
template<std::size_t DOF>
float Robot::RobotArm< DOF >::getJointAngle (
            std::size_t index ) const  [inline]
```

Gets current angle of a single joint.

Parameters

index	Joint index (0 to DOF-1)

Returns

Joint angle in radians

7.3.3.3 getJointAngles()

```
template<std::size_t DOF>
Matrix< DOF, 1 > Robot::RobotArm< DOF >::getJointAngles ( ) const  [inline]
```

Gets current angles of all joints.

Returns

Vector of DOF joint angles in radians

7.3.3.4 goTo()

```
template<std::size_t DOF>
bool Robot::RobotArm< DOF >::goTo (
            const Matrix< 3, 1 > & pr,
            const Matrix< 3, 3 > & Rr )  [inline]
```

Moves robot end-effector to desired position and orientation.

Parameters

pr	Desired end-effector position [x, y, z] in meters
Rr	Desired end-effector rotation matrix (3x3)

Returns

true if IK solution found and successfully moved to target, false if IK failed or joint angles exceeded limits

Solves inverse kinematics and updates joint actuators if solution found. Uses predefined convergence tolerances (1e-5 m position, 1e-3 rad orientation) and Levenberg-Marquardt damping factor 1e-2 with maximum 50 iterations.

7.3.3.5 setJointAngles()

```
template<std::size_t DOF>
bool Robot::RobotArm< DOF >::setJointAngles (
            const Matrix< DOF, 1 > & angles )  [inline]
```

Sets and commits joint angles to hardware actuators.

Parameters

angles	Target joint angles in radians

Returns

true if angles were within limits and successfully written, false if any angle exceeded joint constraints

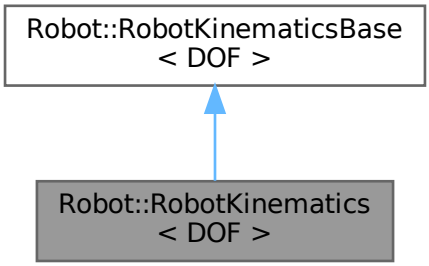
Updates internal state only if actuators successfully write the angles.
The documentation for this class was generated from the following file:

- TP26-Roboticke-rameno/robot.h

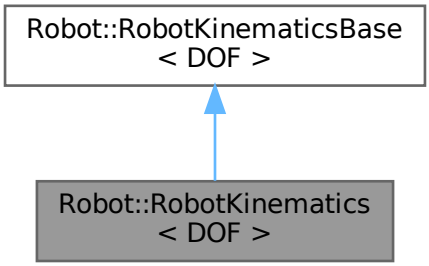
7.4 Robot::RobotKinematics< DOF > Class Template Reference

Concrete kinematics solver using Jacobian-based inverse kinematics.
`#include <robot.h>`

Inheritance diagram for Robot::RobotKinematics< DOF >:



Collaboration diagram for Robot::RobotKinematics< DOF >:



Public Member Functions

- RobotKinematics (const std::array< Matrix< 3, 1 >, DOF > &L_, const std::array< const RotationMatrix *, DOF > &R_)

  Initializes kinematics solver with robot link parameters.
- std::pair< Matrix< 3, 1 >, Matrix< 3, 3 > > forwardKinematics (const Matrix< DOF, 1 > &theta) const override

  Computes end-effector position and orientation from joint angles.
- std::array< Matrix< 3, 3 >, DOF > getRotDerivatives (const Matrix< DOF, 1 > &theta) const

  Computes partial derivatives of rotation matrices with respect to joint angles.
- Matrix< 3, DOF > getJacobian (const Matrix< DOF, 1 > &theta) const

  Computes 3×DOF geometric Jacobian for end-effector linear velocity.
- bool SolveIK (const Matrix< 3, 1 > &pr, const Matrix< 3, 3 > &Rr, Matrix< DOF, 1 > &theta, float tol_pos, float tol_ori, float lambda, int max_iter) override

  Solves inverse kinematics using Levenberg-Marquardt optimization.

7.4.1 Detailed Description

template<std::size_t DOF>
class Robot::RobotKinematics< DOF >

Concrete kinematics solver using Jacobian-based inverse kinematics.

Template Parameters

DOF	Degrees of freedom (number of joints)

Implements forward and inverse kinematics for a robot arm using:

- Forward kinematics: Composition of rotation matrices and translations
- Inverse kinematics: Levenberg-Marquardt numerical optimization with analytical Jacobian computation for both position and orientation errors

7.4.2 Constructor & Destructor Documentation

7.4.2.1 RobotKinematics()

```
template<std::size_t DOF>
Robot::RobotKinematics< DOF >::RobotKinematics (
            const std::array< Matrix< 3, 1 >, DOF > & L_,
            const std::array< const RotationMatrix *, DOF > & R_ )  [inline]
```

Initializes kinematics solver with robot link parameters.

Parameters

*L_*	Array of link translation vectors (DH parameter d)
*R_*	Array of rotation function pointers for each joint Each RotationMatrix* implements R(theta) = rotation matrix for that joint

7.4.3 Member Function Documentation

7.4.3.1 forwardKinematics()

```
template<std::size_t DOF>
std::pair< Matrix< 3, 1 >, Matrix< 3, 3 > > Robot::RobotKinematics< DOF >::forwardKinematics
(
            const Matrix< DOF, 1 > & theta ) const  [inline], [override], [virtual]
```

Computes end-effector position and orientation from joint angles.

Parameters

theta	Joint angles in radians

Returns

Pair of (end-effector position [x,y,z], end-effector rotation matrix)

Implements DH-based forward kinematics: p = (R0 * R1 * ... * Rn) * (L0 + L1 + ...)
Implements Robot::RobotKinematicsBase< DOF >.

7.4.3.2 getJacobian()

```
template<std::size_t DOF>
Matrix< 3, DOF > Robot::RobotKinematics< DOF >::getJacobian (
            const Matrix< DOF, 1 > & theta ) const  [inline]
```

Computes 3×DOF geometric Jacobian for end-effector linear velocity.

Parameters

theta	Current joint angles in radians

Returns

3×DOF Jacobian matrix: dp/dtheta

Relates joint angular velocities to end-effector linear velocity: v_end = J * dtheta/dt
Each column j represents the contribution of joint j's rotation to end-effector velocity.

7.4.3.3 getRotDerivatives()

```
template<std::size_t DOF>
std::array< Matrix< 3, 3 >, DOF > Robot::RobotKinematics< DOF >::getRotDerivatives (
            const Matrix< DOF, 1 > & theta ) const  [inline]
```

Computes partial derivatives of rotation matrices with respect to joint angles.

Parameters

theta	Current joint angles in radians

Returns

Array of DOF matrices: dR_i/dtheta_i for each joint i

Used internally by inverse kinematics to build the Jacobian for orientation error (last 3 rows). Computed using: dE_i = dR_i/dtheta_i * Rr^T where dR_i is the derivative of i-th rotation matrix

7.4.3.4 SolveIK()

```
template<std::size_t DOF>
bool Robot::RobotKinematics< DOF >::SolveIK (
            const Matrix< 3, 1 > & pr,
            const Matrix< 3, 3 > & Rr,
            Matrix< DOF, 1 > & theta,
            float tol_pos,
            float tol_ori,
            float lambda,
            int max_iter )  [inline], [override], [virtual]
```

Solves inverse kinematics using Levenberg-Marquardt optimization.

Parameters

pr	Desired end-effector position [x, y, z] in meters
Rr	Desired end-effector rotation matrix (3×3)

Parameters

theta	[in/out] Starting joint angles (input), IK solution (output)
tol_pos	Position error tolerance in meters
tol_ori	Orientation error tolerance in radians (extracted from R_error)
lambda	Damping factor for Levenberg-Marquardt regularization
max_iter	Maximum iterations before giving up

Returns

true if converged to solution meeting both tolerances, false if max iterations exceeded without convergence

Algorithm:

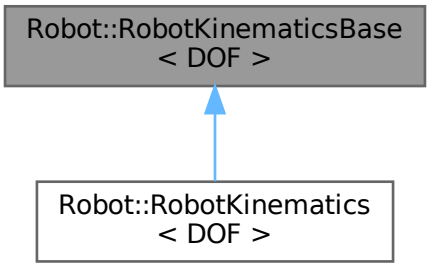
1. Compute forward kinematics for current pose
2. Calculate position error: rp = p_current - p_desired
3. Calculate orientation error from rotation matrix: r0 = [R_err(1,0), R_err(2,0), R_err(2,1)]
4. Build 6×DOF Jacobian combining position and orientation derivatives
5. Apply Levenberg-Marquardt damping to Jacobian diagonal
6. Solve for joint angle updates: delta_theta = J^-1 ∗ (-residuals)
7. Update joint angles and repeat until convergence

Implements Robot::RobotKinematicsBase< DOF >.
The documentation for this class was generated from the following file:

- TP26-Roboticke-rameno/robot.h

7.5 Robot::RobotKinematicsBase< DOF > Class Template Reference

Abstract base class for robot kinematics solvers.
`#include <robot.h>`
Inheritance diagram for Robot::RobotKinematicsBase< DOF >:



Public Member Functions

- virtual bool SolveIK (const Matrix< 3, 1 > &pr, const Matrix< 3, 3 > &Rr, Matrix< DOF, 1 > &theta_, float tol_pos, float tol_ori, float lambda, int max_iter)=0

  Solves inverse kinematics using numerical optimization.
- virtual std::pair< Matrix< 3, 1 >, Matrix< 3, 3 > > forwardKinematics (const Matrix< DOF, 1 > &theta) const =0

  Computes end-effector position and orientation from joint angles.

7.5.1 Detailed Description

template<std::size_t DOF>
class Robot::RobotKinematicsBase< DOF >

Abstract base class for robot kinematics solvers.

Template Parameters

DOF	Degrees of freedom (number of joints)

Defines the interface for forward and inverse kinematics computations. Concrete implementations must solve both FK (joint angles → end-effector pose) and IK (desired pose → joint angles).

7.5.2 Member Function Documentation

7.5.2.1 forwardKinematics()

```
template<std::size_t DOF>
virtual std::pair< Matrix< 3, 1 >, Matrix< 3, 3 > > Robot::RobotKinematicsBase< DOF >↩
::forwardKinematics (
            const Matrix< DOF, 1 > & theta ) const  [pure virtual]
```

Computes end-effector position and orientation from joint angles.

Parameters

theta	Joint angles in radians

Returns

Pair of (position vector [x,y,z], rotation matrix 3x3)

Implemented in Robot::RobotKinematics< DOF >.

7.5.2.2 SolveIK()

```
template<std::size_t DOF>
virtual bool Robot::RobotKinematicsBase< DOF >::SolveIK (
            const Matrix< 3, 1 > & pr,
            const Matrix< 3, 3 > & Rr,
            Matrix< DOF, 1 > & theta_,
            float tol_pos,
            float tol_ori,
            float lambda,
            int max_iter )  [pure virtual]
```

Solves inverse kinematics using numerical optimization.

Parameters

pr	Desired end-effector position [x, y, z] in meters

Parameters

Rr	Desired end-effector rotation matrix (3x3)
*theta_*	[in/out] Joint angles: initialized with starting values, updated with IK solution
tol_pos	Position tolerance in meters (convergence criterion)
tol_ori	Orientation tolerance in radians (convergence criterion)
lambda	Damping factor for Levenberg-Marquardt regularization
max_iter	Maximum number of iterations

Returns

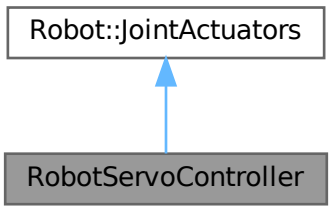
true if solution found within tolerance, false otherwise

Implemented in Robot::RobotKinematics< DOF >.
The documentation for this class was generated from the following file:

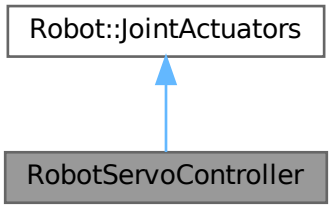
- TP26-Roboticke-rameno/robot.h

7.6 RobotServoController Class Reference

Hardware interface for servo motor control via PCA9685 PWM driver.
`#include <ServoActuator.h>`
Inheritance diagram for RobotServoController:



Collaboration diagram for RobotServoController:



Public Member Functions

- RobotServoController (TwoWire &Wire_, const std::vector< actuator_parameters > &parameters_)

  Initializes servo controller with I2C bus and calibration parameters.
- void init ()

  Initializes PWM driver hardware and configures servo frequency.
- bool writeAngles (const float *angles, std::size_t n)

  Writes joint angles to all servo actuators.

Public Member Functions inherited from Robot::JointActuators

- **JointActuators** ()

  Default constructor.

7.6.1 Detailed Description

Hardware interface for servo motor control via PCA9685 PWM driver.
Implements the JointActuators interface to convert computed joint angles into PWM control signals for servo motors.
Handles:

- Joint angle to PWM conversion with calibration parameters
- Servo frequency initialization and PWM channel configuration
- Joint angle limit enforcement to prevent hardware damage

Each joint requires:

- Minimum and maximum angle constraints
- Calibration gain and offset factors

Works with STM32 and I2C-based PCA9685 16-channel PWM expander for simultaneous multi-servo control with precise timing.

7.6.2 Constructor & Destructor Documentation

7.6.2.1 RobotServoController()

```
RobotServoController::RobotServoController (
            TwoWire & Wire_,
            const std::vector< actuator_parameters > & parameters_ )  [inline]
```

Initializes servo controller with I2C bus and calibration parameters.

Parameters

*Wire_*	Reference to the I2C bus (TwoWire) for PCA9685 communication
*parameters_*	Vector of actuator_parameters (one per joint) containing: • minAngle, maxAngle: joint limits in radians • gain, offset: calibration factors for angle-to-PWM conversion

Does not take ownership of the Wire_ reference; caller must manage the I2C bus. Parameters are stored as a reference; the caller must ensure the vector remains valid for the lifetime of this object.

7.6.3 Member Function Documentation

7.6.3.1 init()

```
void RobotServoController::init ( )  [inline]
```

Initializes PWM driver hardware and configures servo frequency.
Operations performed:

1. Reset all PCA9685 devices on the I2C bus
2. Initialize PCA9685 registers and communication
3. Set PWM frequency to standard servo frequency (typically 50 Hz)

Must be called once during system initialization before sending commands to servos. Typically invoked in the main setup() or initialization task.

7.6.3.2 writeAngles()

```
bool RobotServoController::writeAngles (
            const float * angles,
            std::size_t n )  [inline], [virtual]
```

Writes joint angles to all servo actuators.

Parameters

angles	Array of joint angles in radians
n	Number of joints (array size; should match configured DOF)

Returns

true if all angles were within valid ranges and successfully written to hardware, false if any angle exceeded its joint limits (no angles are written in this case)

Conversion process for each joint i:

1. Check: angle_min $\leq$ angles[i] $\leq$ angle_max
2. If violated: abort and return false (fail-safe behavior)
3. If valid: convert to degrees: $deg\_i = (angles[i] - offset\_i) * gain\_i * 180/\pi$
4. Map degrees to PWM pulse width using servo calibration
5. Write PWM value to PCA9685 channel i

Note

Operates in a fail-safe manner: if ANY angle violates its constraints, no PWM values are written. This prevents partial joint movements that could cause unsafe arm configurations.

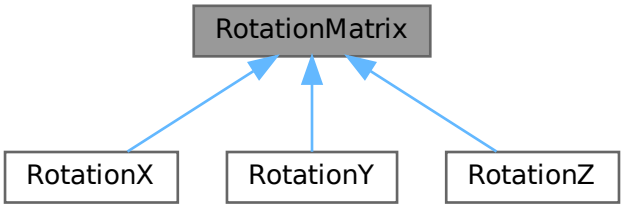
Implements Robot::JointActuators.
The documentation for this class was generated from the following file:

- TP26-Roboticke-rameno/ServoActuator.h

7.7 RotationMatrix Class Reference

Abstract base class for joint rotation matrix generation.
`#include <geometry.h>`

Inheritance diagram for RotationMatrix:



Public Member Functions

- virtual ~**RotationMatrix** ()=default

  Virtual destructor.
- virtual Matrix< 3, 3 > operator() (float theta) const =0

  Compute rotation matrix from angle.
- virtual Matrix< 3, 3 > d (float theta) const =0

  Compute derivative of rotation matrix with respect to angle.
- virtual void operator() (float theta, Matrix< 3, 3 > R, Matrix< 3, 3 > &dR) const =0

  Compute both rotation matrix and derivative in one call.

7.7.1 Detailed Description

Abstract base class for joint rotation matrix generation.
Defines an interface for generating rotation matrices in a fixed axis (X, Y, or Z) and computing their derivatives with respect to joint angles.
Implementations are used in robot kinematics solvers to:

- Compute cumulative rotation transformations
- Calculate Jacobian matrices for inverse kinematics
- Perform forward and inverse kinematic analyses

7.7.2 Member Function Documentation

7.7.2.1 d()

```
virtual Matrix< 3, 3 > RotationMatrix::d (
            float theta ) const  [pure virtual]
```

Compute derivative of rotation matrix with respect to angle.

Parameters

theta	Joint angle in radians

Returns

3×3 derivative matrix: $dR/d\theta$

Implemented in RotationX, RotationY, and RotationZ.

7.7.2.2 operator()() [1/2]

```
virtual Matrix< 3, 3 > RotationMatrix::operator() (
            float theta ) const  [pure virtual]
```

Compute rotation matrix from angle.

Parameters

theta	Joint angle in radians

Returns

3×3 rotation matrix for this joint

Implemented in RotationX, RotationY, and RotationZ.

7.7.2.3 operator()() [2/2]

```
virtual void RotationMatrix::operator() (
            float theta,
            Matrix< 3, 3 > R,
            Matrix< 3, 3 > & dR ) const  [pure virtual]
```

Compute both rotation matrix and derivative in one call.

Parameters

theta	Joint angle in radians
R	[out] Computed rotation matrix (passed by value, updated in call)
dR	[out] Computed derivative matrix (passed by reference)

Note

The parameter R is intentionally passed by value for API consistency. Only dR is modified within this function. This is an optimization pattern where both computations share sin/cos evaluation.

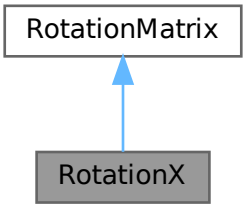
Implemented in RotationX, RotationY, and RotationZ.
The documentation for this class was generated from the following file:

- TP26-Roboticke-rameno/geometry.h

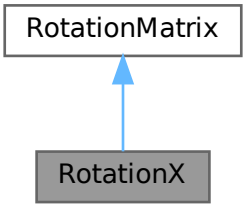
7.8 RotationX Class Reference

Rotation in X-axis (right-hand rule: thumb along +X)
`#include <geometry.h>`

Inheritance diagram for RotationX:



Collaboration diagram for RotationX:



Public Member Functions

- Matrix< 3, 3 > operator() (float theta) const override

  Compute X-axis rotation matrix.
- Matrix< 3, 3 > d (float theta) const override

  Compute derivative of X-axis rotation matrix.
- void operator() (float theta, Matrix< 3, 3 > R, Matrix< 3, 3 > &dR) const override

  Compute both X-axis rotation matrix and derivative.

Public Member Functions inherited from RotationMatrix

- virtual ~**RotationMatrix** ()=default

  Virtual destructor.

7.8.1 Detailed Description

Rotation in X-axis (right-hand rule: thumb along +X)
Concrete implementation of RotationMatrix for rotations in the X-axis. Common for shoulder/base joints in robotic manipulators.

7.8.2 Member Function Documentation

7.8.2.1 d()

```
Matrix< 3, 3 > RotationX::d (
            float theta ) const  [inline], [override], [virtual]
```

Compute derivative of X-axis rotation matrix.

Parameters

theta	Rotation angle in radians

Returns

3×3 derivative matrix: dR_x/dθ

Implements RotationMatrix.

7.8.2.2 operator()() [1/2]

```
Matrix< 3, 3 > RotationX::operator() (
            float theta ) const  [inline], [override], [virtual]
```

Compute X-axis rotation matrix.

Parameters

theta	Rotation angle in radians

Returns

3×3 rotation matrix in X-axis

Implements RotationMatrix.

7.8.2.3 operator()() [2/2]

```
void RotationX::operator() (
            float theta,
            Matrix< 3, 3 > R,
            Matrix< 3, 3 > & dR ) const  [inline], [override], [virtual]
```

Compute both X-axis rotation matrix and derivative.

Parameters

theta	Rotation angle in radians
R	[out] Computed rotation matrix
dR	[out] Computed derivative matrix

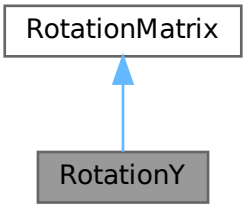
Implements RotationMatrix.
The documentation for this class was generated from the following file:

- TP26-Roboticke-rameno/geometry.h

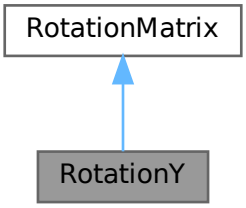
7.9 RotationY Class Reference

Rotation in Y-axis (right-hand rule: thumb along +Y)
`#include <geometry.h>`

Inheritance diagram for RotationY:



Collaboration diagram for RotationY:



Public Member Functions

- Matrix< 3, 3 > operator() (float theta) const override

  Compute Y-axis rotation matrix.
- Matrix< 3, 3 > d (float theta) const override

  Compute derivative of Y-axis rotation matrix.
- void operator() (float theta, Matrix< 3, 3 > R, Matrix< 3, 3 > &dR) const override

  Compute both Y-axis rotation matrix and derivative.

Public Member Functions inherited from RotationMatrix

- virtual ~**RotationMatrix** ()=default

  Virtual destructor.

7.9.1 Detailed Description

Rotation in Y-axis (right-hand rule: thumb along +Y)
Concrete implementation of RotationMatrix for rotations in the Y-axis. Common for elbow and wrist pitch joints in robotic manipulators.

7.9.2 Member Function Documentation

7.9.2.1 d()

```
Matrix< 3, 3 > RotationY::d (
            float theta ) const  [inline], [override], [virtual]
```

Compute derivative of Y-axis rotation matrix.

Parameters

theta	Rotation angle in radians

Returns

3×3 derivative matrix: dR_y/dθ

Implements RotationMatrix.

7.9.2.2 operator()() [1/2]

```
Matrix< 3, 3 > RotationY::operator() (
            float theta ) const  [inline], [override], [virtual]
```

Compute Y-axis rotation matrix.

Parameters

theta	Rotation angle in radians

Returns

3×3 rotation matrix in Y-axis

Implements RotationMatrix.

7.9.2.3 operator()() [2/2]

```
void RotationY::operator() (
            float theta,
            Matrix< 3, 3 > R,
            Matrix< 3, 3 > & dR ) const  [inline], [override], [virtual]
```

Compute both Y-axis rotation matrix and derivative.

Parameters

theta	Rotation angle in radians
R	[out] Computed rotation matrix
dR	[out] Computed derivative matrix

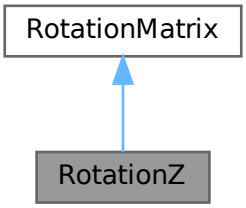
Implements RotationMatrix.
The documentation for this class was generated from the following file:

- TP26-Roboticke-rameno/geometry.h

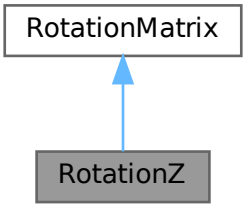
7.10 RotationZ Class Reference

Rotation in Z-axis (right-hand rule: thumb along +Z)
`#include <geometry.h>`

Inheritance diagram for RotationZ:



Collaboration diagram for RotationZ:



Public Member Functions

- Matrix< 3, 3 > operator() (float theta) const override

  Compute Z-axis rotation matrix.
- Matrix< 3, 3 > d (float theta) const override

  Compute derivative of Z-axis rotation matrix.
- void operator() (float theta, Matrix< 3, 3 > R, Matrix< 3, 3 > &dR) const override

  Compute both Z-axis rotation matrix and derivative.

Public Member Functions inherited from RotationMatrix

- virtual ~**RotationMatrix** ()=default

  Virtual destructor.

7.10.1 Detailed Description

Rotation in Z-axis (right-hand rule: thumb along +Z)
Concrete implementation of RotationMatrix for rotations in the Z-axis. Common for waist/azimuth joints and wrist yaw in robotic manipulators.

7.10.2 Member Function Documentation

7.10.2.1 d()

```
Matrix< 3, 3 > RotationZ::d (
            float theta ) const  [inline], [override], [virtual]
```

Compute derivative of Z-axis rotation matrix.

Parameters

theta	Rotation angle in radians

Returns

3×3 derivative matrix: $\mathrm{d}R_z/\mathrm{d}\theta$

Implements RotationMatrix.

7.10.2.2 operator()() [1/2]

```
Matrix< 3, 3 > RotationZ::operator() (
            float theta ) const  [inline], [override], [virtual]
```

Compute Z-axis rotation matrix.

Parameters

theta	Rotation angle in radians

Returns

3×3 rotation matrix in Z-axis

Implements RotationMatrix.

7.10.2.3 operator()() [2/2]

```
void RotationZ::operator() (
            float theta,
            Matrix< 3, 3 > R,
            Matrix< 3, 3 > & dR ) const  [inline], [override], [virtual]
```

Compute both Z-axis rotation matrix and derivative.

Parameters

theta	Rotation angle in radians
R	[out] Computed rotation matrix
dR	[out] Computed derivative matrix

Implements RotationMatrix.
The documentation for this class was generated from the following file:

- TP26-Roboticke-rameno/geometry.h

Chapter 8

File Documentation

8.1 geometry.h

```
00001 #pragma once
00002
00003 #include "Eigen/Dense"
00004
00005
00009 template<int Rows, int Cols>
00010 using Matrix = Eigen::Matrix<float, Rows, Cols>;
00011
00023
00032 Matrix<3,3> rotX(float s_theta, float c_theta)
00033 {
00034     Matrix<3, 3> R;
00035     R << 1.0,     0,      0,
00036          0, c_theta, -s_theta,
00037          0, s_theta,  c_theta;
00038     return R;
00039 }
00040
00046 Matrix<3,3> rotX(float theta)
00047 {
00048     return rotX(sin(theta), cos(theta));
00049 }
00050
00060 Matrix<3,3> drotX(float s_theta, float c_theta)
00061 {
00062     Matrix<3, 3> R;
00063     R << 0, 0, 0,
00064          0, -s_theta, -c_theta,
00065          0,  c_theta, -s_theta;
00066     return R;
00067 }
00068
00074 Matrix<3,3> drotX(float theta)
00075 {
00076     return drotX(sin(theta), cos(theta));
00077 }
00078
00087 Matrix<3,3> rotY(float s_theta, float c_theta)
00088 {
00089     Matrix<3, 3> R;
00090     R << c_theta, 0, s_theta,
00091          0,      1.0, 0,
00092         -s_theta, 0, c_theta;
00093     return R;
00094 }
00095
00101 Matrix<3,3> rotY(float theta)
00102 {
00103     return rotY(sin(theta), cos(theta));
00104 }
00105
00115 Matrix<3,3> drotY(float s_theta, float c_theta)
00116 {
00117     Matrix<3, 3> R;
00118     R << -s_theta, 0, c_theta,
00119           0,      0, 0,
00120          -c_theta, 0, -s_theta;
00121     return R;
00122 }
00123
00129 Matrix<3,3> drotY(float theta)
00130 {
```

```
00131     return drotY(sin(theta), cos(theta));
00132 }
00133
00142 Matrix<3,3> rotZ(float s_theta, float c_theta)
00143 {
00144     Matrix<3, 3> R;
00145     R << c_theta, -s_theta, 0,
00146          s_theta,  c_theta, 0,
00147          0,        0,       1.0;
00148     return R;
00149 }
00150
00156 Matrix<3,3> rotZ(float theta)
00157 {
00158     return rotZ(sin(theta), cos(theta));
00159 }
00160
00170 Matrix<3,3> drotZ(float s_theta, float c_theta)
00171 {
00172     Matrix<3, 3> R;
00173     R << -s_theta, -c_theta, 0,
00174           c_theta, -s_theta, 0,
00175           0,        0,       0;
00176     return R;
00177 }
00178
00184 Matrix<3,3> drotZ(float theta)
00185 {
00186     return drotZ(sin(theta), cos(theta));
00187 }
00188
00189
00202 class RotationMatrix {
00203 public:
00207     virtual ~RotationMatrix() = default;
00208
00214     virtual Matrix<3,3> operator()(float theta) const = 0;
00215
00221     virtual Matrix<3,3> d(float theta) const = 0;
00222
00233     virtual void operator()(float theta, Matrix<3,3> R, Matrix<3,3> &dR) const = 0;
00234 };
00235
00243 class RotationX : public RotationMatrix {
00244 public:
00250     Matrix<3,3> operator()(float theta) const override {
00251         float s = sin(theta), c = cos(theta);
00252         return rotX(s, c);
00253     }
00254
00260     Matrix<3,3> d(float theta) const override {
00261         float s = sin(theta), c = cos(theta);
00262         return drotX(s, c);
00263     }
00264
00271     void operator()(float theta, Matrix<3,3> R, Matrix<3,3> &dR) const override {
00272         float s = sin(theta), c = cos(theta);
00273         R = rotX(s, c);
00274         dR = drotX(s, c);
00275     }
00276 };
00277
00285 class RotationY : public RotationMatrix {
00286 public:
00292     Matrix<3,3> operator()(float theta) const override {
00293         float s = sin(theta), c = cos(theta);
00294         return rotY(s, c);
00295     }
00296
00302     Matrix<3,3> d(float theta) const override {
00303         float s = sin(theta), c = cos(theta);
00304         return drotY(s, c);
00305     }
00306
00313     void operator()(float theta, Matrix<3,3> R, Matrix<3,3> &dR) const override {
00314         float s = sin(theta), c = cos(theta);
00315         R = rotY(s, c);
00316         dR = drotY(s, c);
00317     }
00318 };
00319
00327 class RotationZ : public RotationMatrix {
00328 public:
00334     Matrix<3,3> operator()(float theta) const override {
00335         float s = sin(theta), c = cos(theta);
00336         return rotZ(s, c);
00337     }
```

```
00338
00344     Matrix<3,3> d(float theta) const override {
00345         float s = sin(theta), c = cos(theta);
00346         return drotZ(s, c);
00347     }
00348
00355     void operator()(float theta, Matrix<3,3> R, Matrix<3,3> &dR) const override {
00356         float s = sin(theta), c = cos(theta);
00357         R = rotZ(s, c);
00358         dR = drotZ(s, c);
00359     }
00360 };
00361
00362
00363
00364
```

8.2 robot.h

```
00001 #pragma once
00002 #include <cstddef>
00003 #include <array>
00004 #include "geometry.h"
00005
00006 template<int Rows, int Cols>
00007 using Matrix = Eigen::Matrix<float, Rows, Cols>;
00008
00016 typedef struct
00017 {
00018   float minAngle;
00019   float maxAngle;
00020   float gain;
00021   float offset;
00022 } actuator_parameters;
00023
00024
00032 namespace Robot {
00033
00041   static inline Matrix<3,1> T(float l) {
00042     Matrix<3,1> t;
00043     t « 0.0f, 0.0f, l;
00044     return t;
00045   }
00046
00056 template<std::size_t DOF>
00057 class RobotKinematicsBase {
00058 public:
00059
00072   virtual bool SolveIK(
00073     const Matrix<3, 1>& pr,
00074     const Matrix<3, 3>& Rr,
00075     Matrix<DOF, 1>& theta_,
00076     float tol_pos,
00077     float tol_ori,
00078     float lambda,
00079     int max_iter)  = 0;
00080
00086   virtual std::pair<Matrix<3, 1>, Matrix<3, 3» forwardKinematics(const Matrix<DOF, 1>& theta) const =
      0;
00087 };
00088
00089
00090
00099 class JointActuators {
00100 public:
00104   JointActuators() {}
00105
00113   virtual bool writeAngles(const float*, std::size_t) = 0;
00114 };
00115
00126 template<std::size_t DOF>
00127 class RobotArm {
00128 public:
00129
00137   RobotArm(RobotKinematicsBase<DOF>* robot_kinematics, JointActuators* actuators)
00138     : robot_kinematics_(robot_kinematics), jointAngles_(Matrix<DOF, 1>::Zero()), actuators_(actuators)
      {
00139   }
00140
00144   ~RobotArm() = default;
00145
00146
00155   bool setJointAngles(const Matrix<DOF, 1>& angles) {
00156
00157     if (!actuators_->writeAngles(angles.data(), DOF)) return false;
```

```
00158     jointAngles_ = angles;
00159     return true;
00160   }
00161
00167   float getJointAngle(std::size_t index) const {
00168     return jointAngles_(index);
00169   }
00170
00175   Matrix<DOF, 1> getJointAngles() const {
00176     return jointAngles_;
00177   }
00178
00184   std::pair<Matrix<3, 1>, Matrix<3, 3» getCurrentEffectorPositionOrientation() const {
00185     return robot_kinematics_->forwardKinematics(jointAngles_);
00186   }
00187
00199   bool goTo(const Matrix<3, 1>& pr, const Matrix<3, 3>& Rr) {
00200     Matrix<DOF, 1> theta = jointAngles_;
00201     bool res = robot_kinematics_->SolveIK(pr, Rr, theta, 1e-5f, 1e-3f, 1e-2, 50);
00202
00203     if (res) {
00204       res = setJointAngles(theta);
00205     }
00206     return res;
00207   }
00208
00209
00210
00211 protected:
00212   JointActuators* actuators_;
00213   Matrix<DOF, 1> jointAngles_;
00214   RobotKinematicsBase<DOF>* robot_kinematics_;
00215 };
00216
00227 template<std::size_t DOF>
00228 class RobotKinematics : public RobotKinematicsBase<DOF>
00229 {
00230 public:
00231
00238   RobotKinematics(const std::array<Matrix<3, 1>, DOF>& L_,
00239                   const std::array<const RotationMatrix*, DOF>& R_)
00240       : L(L_), R_func(R_) {}
00241
00249   std::pair<Matrix<3,1>, Matrix<3,3»
00250   forwardKinematics(const Matrix<DOF,1>& theta) const override
00251   {
00252         // Iterujeme od posledného kĺbu dovnútra smerom k základni
00253         Matrix<3,1> vekt = Matrix<3,1>::Zero();
00254         for (int i = (int)DOF - 1; i >= 0; --i)
00255         {
00256             vekt = (*R_func[i])(theta(i)) * (L[i] + vekt);
00257         }
00258
00259         // Orientácia efektora: R1 * R2 * ... * Rn
00260         Matrix<3,3> R_ee = Matrix<3,3>::Identity();
00261         for (std::size_t i = 0; i < DOF; ++i)
00262             R_ee = R_ee * (*R_func[i])(theta(i));
00263
00264         return { vekt, R_ee };
00265     }
00266
00276   std::array<Matrix<3, 3>, DOF> getRotDerivatives(const Matrix<DOF, 1>& theta) const {
00277     std::array<Matrix<3, 3>, DOF> mat_all;         // Pole pre výsledné parciálne derivácie
00278     int n = DOF;
00279     Matrix<3, 3> f_mat = Matrix<3, 3>::Identity();
00280     Matrix<3, 3> b_mat = Matrix<3, 3>::Identity();
00281
00282     // Prvý cyklus: Výpočet celkového dopredného súčinu matíc
00283     // Týmto f_mat naakumuluje súčin R_0 * R_1 * ... * R_{n-2}
00284     for (int i = n - 2; i >= 0; i--) {
00285         f_mat = (*R_func[i])(theta(i)) * f_mat;
00286     }
00287
00288     // Druhý cyklus: Výpočet derivácií smerom od konca k začiatku
00289     for (int j = n - 1; j >= 0; j--) {
00290         if (j < n - 1) {
00291             b_mat = (*R_func[j + 1])(theta(j + 1)) * b_mat;
00292
00293             // "Odstraňovanie" aktuálnej matice zľava pomocou transpozície (R^-1 = R^T)
00294             f_mat = f_mat * (*R_func[j])(theta(j)).transpose();
00295         }
00296
00297         // Samotný výpočet derivácie pre j-ty kĺb
00298         // Pre prístup k derivácii voláme metódu d() cez pointer z tvojho poľa R_func
00299         mat_all[j] = f_mat * R_func[j]->d(theta(j)) * b_mat;
00300     }
00301
```

```
00302     return mat_all;
00303 }
00304
00316   Matrix<3, DOF> getJacobian(const Matrix<DOF, 1>& theta) const {
00317     int n = DOF; // Počet kĺbov
00318     Matrix<3, DOF> jakob_mat = Matrix<3, DOF>::Zero(); // Výsledná Jakobián matica (3 riadky, n
      stĺpcov)
00319
00320     Matrix<3, 3> matica = Matrix<3, 3>::Identity(); // [1 0 0; 0 1 0; 0 0 1]
00321     Matrix<3, 1> pom_vekt = L[n - 1];            // Inicializácia T[n] v pseudokóde
00322
00323     // Prvý cyklus: od i = n-1 do 1 (1-based) -> od i = n-2 po 0 (0-based)
00324     // Kumuluje doprednú kinematiku (súčin rotácií od n-2 po 0)
00325     for (int i = n - 2; i >= 0; i--) {
00326         matica = (*R_func[i])(theta(i)) * matica;
00327     }
00328
00329     // Druhý cyklus: od i = n do 1 (1-based) -> od i = n-1 po 0 (0-based)
00330     for (int i = n - 1; i >= 0; i--) {
00331
00332         Matrix<3, 1> vekt = L[n - 1]; // T[n] z pseudokódu
00333
00334         if (i < n - 1) { // Podmienka (n > i) z pseudokódu v 0-based svete
00335
00336             // "Odstraňovanie" rotácie pre aktuálny kĺb
00337             matica = matica * (*R_func[i])(theta(i)).transpose();
00338
00339             // Akumulácia translačných vektorov od konca (efektora) ku kĺbu 'i'
00340             pom_vekt = L[i] + (*R_func[i + 1])(theta(i + 1)) * pom_vekt;
00341
00342             vekt = pom_vekt;
00343         }
00344
00345         // Výpočet samotného stĺpca Jakobiánu
00346         // matica: (R_1 * R_2 ... * R_{i-1})
00347         // dR_i: parciálna derivácia i-teho kĺbu
00348         // vekt: (T_i + R_{i+1} * (T_{i+1} + ...))
00349         vekt = matica * R_func[i]->d(theta(i)) * vekt;
00350
00351         // Uloženie výsledného vektora do i-teho stĺpca matice Jakobiánu
00352         jakob_mat.col(i) = vekt;
00353     }
00354
00355     return jakob_mat;
00356 }
00357
00379   bool SolveIK(
00380       const Matrix<3, 1>& pr,
00381       const Matrix<3, 3>& Rr,
00382       Matrix<DOF, 1>& theta,
00383       float tol_pos,
00384       float tol_ori,
00385       float lambda,
00386       int max_iter) override
00387     {
00388         for (int i = 0; i < max_iter; i++)
00389         {
00390             // 1. Dopredná kinematika (aktuálny stav)
00391             auto fk_res = forwardKinematics(theta);
00392             Matrix<3, 1> p0 = fk_res.first;
00393             Matrix<3, 3> R0 = fk_res.second;
00394
00395             // 2. Výpočet reziduí (chyby)
00396             Matrix<3, 1> rp = p0 - pr;
00397
00398             // Výpočet matice chyby orientácie E = R0 * Rr^T
00399             Matrix<3, 3> E = R0 * Rr.transpose();
00400
00401             // Extrakcia chyby orientácie podl'a tvojho pseudokódu
00402             Matrix<3, 1> r0;
00403             r0(0) = E(1, 0);
00404             r0(1) = E(2, 0);
00405             r0(2) = E(2, 1);
00406
00407             // 3. Kontrola ukončenia (ak sme v tolerancii)
00408             if (rp.norm() < tol_pos && r0.norm() < tol_ori) {
00409                 return true;
00410             }
00411
00412             // 4. Získanie Jakobiánov (tieto funkcie už v robot.h máš)
00413             Matrix<3, DOF> J_p = getJacobian(theta);
00414             std::array<Matrix<3, 3>, DOF> dEr_R = getRotDerivatives(theta);
00415
00416             // Zostavenie plného Jakobiánu 6xDOF
00417             Matrix<6, DOF> J = Matrix<6, DOF>::Zero();
00418
00419             // Horná časť (poloha)
```

```
00420             J.template block<3, DOF>(0, 0) = J_p;
00421
00422             // Dolná časť (orientácia - podľa dE_j = dR_j * Rr^T)
00423             for (std::size_t j = 0; j < DOF; j++) {
00424                 Matrix<3, 3> dE_j = dEr_R[j] * Rr.transpose();
00425
00426                 J(3, j) = dE_j(1, 0);
00427                 J(4, j) = dE_j(2, 0);
00428                 J(5, j) = dE_j(2, 1);
00429             }
00430
00431             // 5. Regularizácia (Levenberg-Marquardt štýl)
00432             for (std::size_t k = 0; k < DOF; k++) {
00433                 J(k, k) += lambda;
00434             }
00435
00436             // 6. Zostavenie vektora záporných reziduí
00437             Matrix<6, 1> minus_r;
00438             minus_r.template block<3, 1>(0, 0) = -rp;
00439             minus_r.template block<3, 1>(3, 0) = -r0;
00440
00441             // 7. Výpočet zmeny uhlov (Riešenie sústavy J * delta_theta = minus_r)
00442             // Pre 6x6 maticu je ColPivHouseholderQR vel'mi stabilná vol'ba
00443             Matrix<DOF, 1> delta_theta = J.colPivHouseholderQr().solve(minus_r);
00444
00445             // 8. Update
00446             theta += delta_theta;
00447         }
00448
00449         return false; // Nepodarilo sa nájst' riešenie v rámci max_iter
00450     }
00451
00452
00453 private:
00454   std::array<Matrix<3, 1>, DOF> L;
00455   std::array<const RotationMatrix*, DOF> R_func;
00456 };
00457
00458 } // namespace Robot
```

8.3 ServoActuator.h

```
00001 #include <vector>
00002
00003 #pragma once
00004
00005 #include "robot.h"
00006 #include "src\PCA9685\PCA9685.h"
00007
00025 class RobotServoController : public Robot::JointActuators {
00026 public:
00027
00039   RobotServoController(TwoWire& Wire_, const std::vector<actuator_parameters>& parameters_)
00040     : Robot::JointActuators(), pwmController(Wire_), parameters(parameters_) {
00041     }
00042
00054   void init() {
00055     pwmController.resetDevices();
00056     pwmController.init();
00057     pwmController.setPWMFreqServo();
00058   }
00059
00078   bool writeAngles(const float* angles, std::size_t n) {
00079     for (int i = 0; i < n; i++) {
00080       if (angles[i] > parameters[i].maxAngle || angles[i] < parameters[i].minAngle) return false;
00081       float degrees = (angles[i] - parameters[i].offset) * parameters[i].gain * 180.0f / PI;
00082       pwmController.setChannelPWM(i, pwmServo.pwmForAngle(degrees));
00083     }
00084     return true;
00085   }
00086
00087
00088 private:
00089   PCA9685 pwmController;
00090   PCA9685_ServoEval pwmServo;
00091   std::vector<actuator_parameters> parameters;
00092 };
00093
```

8.4 STM32FreeRTOSConfig.h

```
00001
```

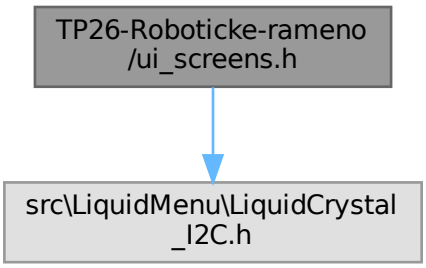
```
00002 #ifndef STM32FREERTOS_CONFIG_H
00003 #define STM32FREERTOS_CONFIG_H
00004
00005 /* Begin custom definitions for STM32 */
00006 /* Define memory allocation implementations to use:
00007  * 1 to 5 for heap_[1-5].c
00008  * -1 for heap_useNewlib_ST.c
00009  * Default -1 see heap.c
00010  */
00011 #define configMEMMANG_HEAP_NB             -1
00012
00013 /* Set to 1 to use default blink hook if configUSE_MALLOC_FAILED_HOOK is 1 */
00014 #ifndef configUSE_MALLOC_FAILED_HOOK_BLINK
00015   #define configUSE_MALLOC_FAILED_HOOK_BLINK  0
00016 #endif
00017 /* Set to 1 to used default blink if configCHECK_FOR_STACK_OVERFLOW is 1 or 2 */
00018 #ifndef configCHECK_FOR_STACK_OVERFLOW_BLINK
00019   #define configCHECK_FOR_STACK_OVERFLOW_BLINK 0
00020 #endif
00021
00022 /* configUSE_CMSIS_RTOS_V2 has to be defined and set to 1 to use CMSIS-RTOSv2 */
00023 /*#define configUSE_CMSIS_RTOS_V2           1*/
00024
00025 /* End custom definitions for STM32 */
00026
00027 /* Ensure stdint is only used by the compiler, and not the assembler. */
00028 #if defined(__ICCARM__) || defined(__CC_ARM) || defined(__GNUC__)
00029  #include <stdint.h>
00030  extern uint32_t SystemCoreClock;
00031 #endif
00032
00033 #if defined(configUSE_CMSIS_RTOS_V2) && (configUSE_CMSIS_RTOS_V2 == 1)
00034 /*------------- CMSIS-RTOS V2 specific defines -----------*/
00035 /* When using CMSIS-RTOSv2 set configSUPPORT_STATIC_ALLOCATION to 1
00036  * is mandatory to avoid compile errors.
00037  * CMSIS-RTOS V2 implmentation requires the following defines
00038  */
00039 /* cmsis_os threads are created using xTaskCreateStatic() API */
00040 #define configSUPPORT_STATIC_ALLOCATION   0
00041 /*  CMSIS-RTOSv2 defines 56 levels of priorities. To be able to use them
00042  *  all and avoid application misbehavior, configUSE_PORT_OPTIMISED_TASK_SELECTION
00043  *  must be set to 0 and configMAX_PRIORITIES to 56
00044  *
00045  */
00046 #define configMAX_PRIORITIES              (56)
00047 /*
00048  * When set to 1, configMAX_PRIORITIES can't be more than 32
00049  * which is not suitable for the new CMSIS-RTOS v2 priority range
00050  */
00051 #define configUSE_PORT_OPTIMISED_TASK_SELECTION 0
00052 /* Require to be a value as used in cmsis_os2.c as array size */
00053 #ifndef configMINIMAL_STACK_SIZE
00054 #define configMINIMAL_STACK_SIZE          ((uint16_t)128)
00055 #endif
00056
00057 #if !defined(CMSIS_device_header)
00058 /* CMSIS_device_header defined to stm32_def.h by default, which include <device.h> like stm32f1xx.h */
00059 #define CMSIS_device_header  "stm32_def.h"
00060 #endif /* CMSIS_device_header */
00061 #else
00062 #define configMAX_PRIORITIES              (7)
00063 #endif /* configUSE_CMSIS_RTOS_V2 */
00064
00065 extern char _end; /* Defined in the linker script */
00066 extern char _estack; /* Defined in the linker script */
00067 extern char _Min_Stack_Size; /* Defined in the linker script */
00068 /*
00069  * _Min_Stack_Size is often set to 0x400 in the linker script
00070  * Use it divided by 8 to set minmimal stack size of a task to 128 by default.
00071  * End user will have to properly configure those value depending to their needs.
00072  */
00073 #ifndef configMINIMAL_STACK_SIZE
00074 #define configMINIMAL_STACK_SIZE          ((uint16_t)((uint32_t)&_Min_Stack_Size/8))
00075 #endif
00076 #ifndef configTOTAL_HEAP_SIZE
00077 #define configTOTAL_HEAP_SIZE             ((size_t)((uint32_t)&_estack - (uint32_t)&_Min_Stack_Size -
      (uint32_t)&_end))
00078 #endif
00079 #ifndef configISR_STACK_SIZE_WORDS
00080 #define configISR_STACK_SIZE_WORDS        ((uint32_t)&_Min_Stack_Size/4)
00081 #endif
00082
00083 #define configUSE_PREEMPTION              1
00084 #define configUSE_IDLE_HOOK               1
00085 #define configUSE_TICK_HOOK               1
00086 #define configCPU_CLOCK_HZ                (SystemCoreClock)
00087 #define configTICK_RATE_HZ                ((TickType_t)1000)
```

```
00088 #define configMAX_TASK_NAME_LEN           (16)
00089 #define configUSE_TRACE_FACILITY          0
00090 #define configUSE_16_BIT_TICKS            0
00091 #define configIDLE_SHOULD_YIELD           1
00092 #define configUSE_MUTEXES                 1
00093 #define configQUEUE_REGISTRY_SIZE         8
00094 #define configCHECK_FOR_STACK_OVERFLOW    0
00095 #define configUSE_RECURSIVE_MUTEXES       0
00096 #define configUSE_MALLOC_FAILED_HOOK      0
00097 #define configUSE_APPLICATION_TASK_TAG    0
00098 #define configUSE_COUNTING_SEMAPHORES     0
00099 #define configGENERATE_RUN_TIME_STATS     0
00100 /*
00101  * If configUSE_NEWLIB_REENTRANT is set to 1 then a newlib reent structure
00102  * will be allocated for each created task.
00103  *
00104  * Note Newlib support has been included by popular demand, but is not used
00105  * by the FreeRTOS maintainers themselves. FreeRTOS is not responsible for
00106  * resulting newlib operation. User must be familiar with newlib and must
00107  * provide system-wide implementations of the necessary stubs.
00108  * Be warned that (at the time of writing) the current newlib design implements
00109  * a system-wide malloc() that must be provided with locks.
00110  */
00111 #define configUSE_NEWLIB_REENTRANT        1
00112
00113 #define configUSE_TIME_SLICING 1
00114
00115 /* Set configENABLE_MPU to 1 to enable MPU support and 0 to disable it. This is
00116 currently used in ARMv8M ports. */
00117 #define configENABLE_MPU                  0
00118 /* Set configENABLE_FPU to 1 to enable FPU support and 0 to disable it. This is
00119 currently used in ARMv8M ports. */
00120 #define configENABLE_FPU                  1
00121 /* Set configENABLE_TRUSTZONE to 1 enable TrustZone support and 0 to disable it.
00122 This is currently used in ARMv8M ports. */
00123 #define configENABLE_TRUSTZONE            0
00124
00125 /* Co-routine definitions. */
00126 #define configUSE_CO_ROUTINES             0
00127 #define configMAX_CO_ROUTINE_PRIORITIES   (2)
00128
00129 /* Software timer definitions. */
00130 #define configUSE_TIMERS                  1
00131 #define configTIMER_TASK_PRIORITY         (5)
00132 #define configTIMER_QUEUE_LENGTH          10
00133 #define configTIMER_TASK_STACK_DEPTH      (configMINIMAL_STACK_SIZE )
00134
00135 /* Set the following definitions to 1 to include the API function, or zero
00136 to exclude the API function. */
00137 #define INCLUDE_vTaskPrioritySet          1
00138 #define INCLUDE_uxTaskPriorityGet         1
00139 #define INCLUDE_vTaskDelete               1
00140 #define INCLUDE_vTaskCleanUpResources     1
00141 #define INCLUDE_vTaskSuspend              1
00142 #define INCLUDE_vTaskDelayUntil           1
00143 #define INCLUDE_vTaskDelay                1
00144 #define INCLUDE_xTaskGetSchedulerState    1
00145 #define INCLUDE_uxTaskGetStackHighWaterMark 1
00146 #define INCLUDE_xTaskGetIdleTaskHandle    1
00147
00148 #if defined(configUSE_CMSIS_RTOS_V2) && (configUSE_CMSIS_RTOS_V2 == 1)
00149 #define INCLUDE_xSemaphoreGetMutexHolder  1
00150 #define INCLUDE_eTaskGetState             1
00151 #define INCLUDE_xTimerPendFunctionCall    1
00152 #define INCLUDE_xTaskGetCurrentTaskHandle 1
00153 #endif
00154
00155 /* Cortex-M specific definitions. */
00156 #ifdef __NVIC_PRIO_BITS
00157  /* __BVIC_PRIO_BITS will be specified when CMSIS is being used. */
00158  #define configPRIO_BITS                  __NVIC_PRIO_BITS
00159 #else
00160  #define configPRIO_BITS                  4        /* 15 priority levels */
00161 #endif
00162
00163 /* The lowest interrupt priority that can be used in a call to a "set priority"
00164 function. */
00165 #define configLIBRARY_LOWEST_INTERRUPT_PRIORITY   0xf
00166
00167 /* The highest interrupt priority that can be used by any interrupt service
00168 routine that makes calls to interrupt safe FreeRTOS API functions.  DO NOT CALL
00169 INTERRUPT SAFE FREERTOS API FUNCTIONS FROM ANY INTERRUPT THAT HAS A HIGHER
00170 PRIORITY THAN THIS! (higher priorities are lower numeric values. */
00171 #define configLIBRARY_MAX_SYSCALL_INTERRUPT_PRIORITY 5
00172
00173 /* Interrupt priorities used by the kernel port layer itself.  These are generic
00174 to all Cortex-M ports, and do not rely on any particular library functions. */
```

```
00175 /* Warning in case of Ethernet, this prio (used by systick) should be higher (lower value) than
      ethernet Timer */
00176 #define configKERNEL_INTERRUPT_PRIORITY   ( 14 << (8 - configPRIO_BITS) )
00177
00178 /* !!!! configMAX_SYSCALL_INTERRUPT_PRIORITY must not be set to zero !!!!
00179 See http://www.FreeRTOS.org/RTOS-Cortex-M3-M4.html. */
00180 #define configMAX_SYSCALL_INTERRUPT_PRIORITY  ( configLIBRARY_MAX_SYSCALL_INTERRUPT_PRIORITY << (8 -
      configPRIO_BITS) )
00181
00182 /* Normal assert() semantics without relying on the provision of an assert.h
00183 header file. */
00184 #define configASSERT( x ) if( ( x ) == 0 ) { taskDISABLE_INTERRUPTS(); for( ;; ); }
00185
00186 /* Definitions that map the FreeRTOS port interrupt handlers to their CMSIS
00187    standard names. */
00188 #define vPortSVCHandler    SVC_Handler
00189 #define xPortPendSVHandler PendSV_Handler
00190
00191 /*
00192  * IMPORTANT:
00193  * SysTick_Handler() from stm32duino core is calling weak osSystickHandler().
00194  * Both CMSIS-RTOSv2 and CMSIS-RTOS override osSystickHandler()
00195  * which is calling xPortSysTickHandler(), defined in respective CortexM-x port
00196  */
00197 /* #define xPortSysTickHandler SysTick_Handler */
00198
00199 #endif /* FREERTOS_CONFIG_DEFAULT_H */
00200
```

8.5 TP26-Roboticke-rameno/ui_screens.h File Reference

User interface display functions for LCD output.
`#include "src\LiquidMenu\LiquidCrystal_I2C.h"`
Include dependency graph for ui_screens.h:



Functions

- void drawXYZ (LiquidCrystal_I2C &lcd, float x, float y, float z)

  Displays end-effector position (X, Y, Z) on LCD screen.
- void drawRPY (LiquidCrystal_I2C &lcd, float r, float p, float y)

  Displays end-effector orientation (Roll-Pitch-Yaw) on LCD screen.

8.5.1 Detailed Description

User interface display functions for LCD output.
Provides inline utility functions for rendering robot arm state information on a 16×2 character LCD display via I2C connection (LiquidCrystal_I2C).
Functions display:

- End-effector position (X, Y, Z Cartesian coordinates)

- End-effector orientation (Roll-Pitch-Yaw Euler angles)

All values are formatted to 1 decimal place for readability on small displays.

8.5.2 Function Documentation

8.5.2.1 drawRPY()

```
void drawRPY (
            LiquidCrystal_I2C & lcd,
            float r,
            float p,
            float y )  [inline]
```

Displays end-effector orientation (Roll-Pitch-Yaw) on LCD screen.

Parameters

lcd	Reference to LiquidCrystal_I2C display object
r	Roll angle in radians (rotation about X-axis)
p	Pitch angle in radians (rotation about Y-axis)
y	Yaw angle in radians (rotation about Z-axis)

Display layout (16×2 character LCD):
```
Row 0: "R:rrr.r  P:ppp.p"
Row 1: "Y:yyy.y"
```
All angles are displayed with 1 decimal place precision. Screen is cleared before rendering.
Rotation convention (right-hand rule):

- Roll (R): Rotation about X-axis (shoulder tilt)
- Pitch (P): Rotation about Y-axis (elbow elevation)
- Yaw (Y): Rotation about Z-axis (wrist rotation)

Note

Used during manual control and end-effector orientation monitoring modes. Angles are typically stored internally in radians but displayed here as-is for direct verification against joint sensor values. Consider converting to degrees for user-friendlier display if needed.

8.5.2.2 drawXYZ()

```
void drawXYZ (
            LiquidCrystal_I2C & lcd,
            float x,
            float y,
            float z )  [inline]
```

Displays end-effector position (X, Y, Z) on LCD screen.

Parameters

lcd	Reference to LiquidCrystal_I2C display object
x	End-effector X position in meters
y	End-effector Y position in meters
z	End-effector Z position in meters

Display layout (16×2 character LCD):
```
Row 0: "X:xxx.x  Y:yyy.y"
Row 1: "Z:zzz.z"
```

All coordinates are displayed with 1 decimal place precision. Screen is cleared before rendering.

Note

Used during manual control and end-effector monitoring modes. Typical refresh rate: 100-500 ms (depends on control loop frequency).

8.6 ui_screens.h

Go to the documentation of this file.

```
00001 #pragma once
00002
00003 #include "src\LiquidMenu\LiquidCrystal_I2C.h"
00004
00038 inline void drawXYZ(LiquidCrystal_I2C& lcd, float x, float y, float z) {
00039   lcd.clear();
00040
00041   lcd.setCursor(0,0);
00042   lcd.print("X:");
00043   lcd.print(x,1);
00044
00045   lcd.setCursor(8,0);
00046   lcd.print("Y:");
00047   lcd.print(y,1);
00048
00049   lcd.setCursor(0,1);
00050   lcd.print("Z:");
00051   lcd.print(z,1);
00052 }
00053
00080 inline void drawRPY(LiquidCrystal_I2C& lcd, float r, float p, float y) {
00081   lcd.clear();
00082
00083   lcd.setCursor(0,0);
00084   lcd.print("R:");
00085   lcd.print(r,1);
00086
00087   lcd.setCursor(8,0);
00088   lcd.print("P:");
00089   lcd.print(p,1);
00090
00091   lcd.setCursor(0,1);
00092   lcd.print("Y:");
00093   lcd.print(y,1);
00094 }
```

Index